# EC311 Introduction to Logic Design Spring 2024
# Seven Segment Display Worksheet

**Goal**: Build a HEX counter using the 7-segment displays which increments every second. The counter shall count up to the largest 16-bit number.

**Step 0**: Download the zip file from blackboard and unzip. You might end up having to copy and paste into Vivado. These files implement a clock divider, decoder, a faster clock divider, an FSM, and a counter. **The counter is the top-level module to be loaded onto the board. It depends on the other modules**. These files have commented-out sections that you have to fill out. Please use the provided testbenches to test your implementation.

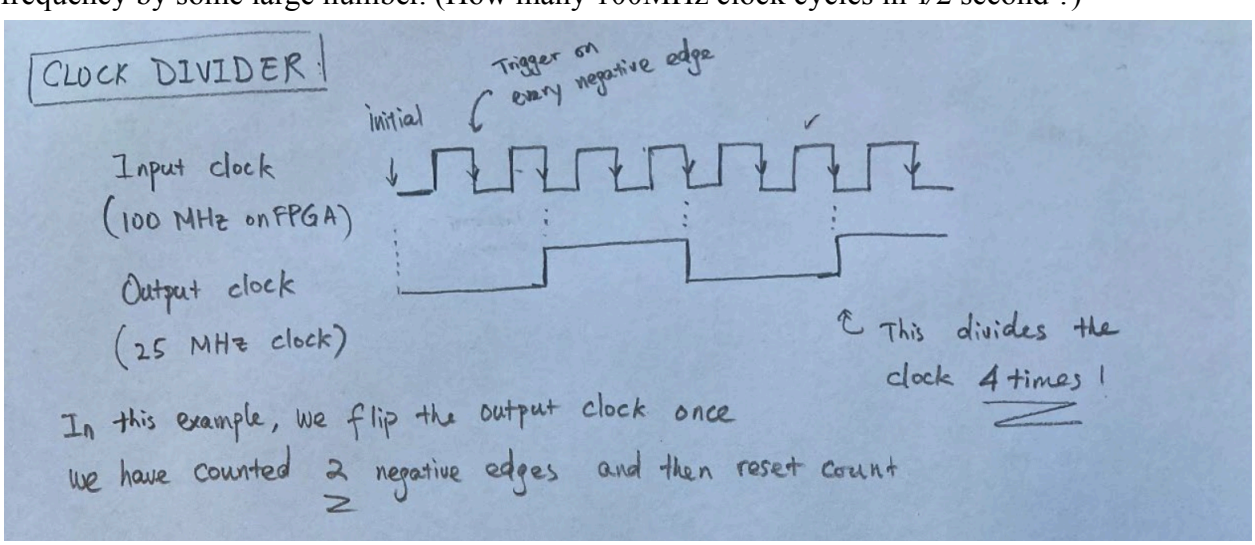
**Step 1**: Build a clock divider

A clock divider divides the frequency of the input clock. A lower-frequency clock may be necessary to drive higher latency components.

(a) Fill in the code for *clock_divider.v*, following the commented instructions. Reference the Figure. Remember that if you flip the output clock after every 2 negative edges, then you divide the clock frequency by 4, not 2.
   (i) **Blocking vs. non-blocking assignments**. In an always block, "=" means blocking, "<=" means non-blocking. Blocking assignments execute sequentially, and non- blocking assignments execute simultaneously. As a rule-of-thumb, use non- blocking assignments unless simultaneous execution breaks your code.

(b) To test your code, make your clock divider divide input clock frequency by 4 and simulate using *clock_divider_test.v.* Check the simulation waveforms against the Figure.

(c) After checking that the testbench works, change the clock divider to divide the clock frequency by some large number. (How many 100MHz clock cycles in 1⁄2 second ?)



Step 2: Build the 7-segment display Finite State Machine (FSM).

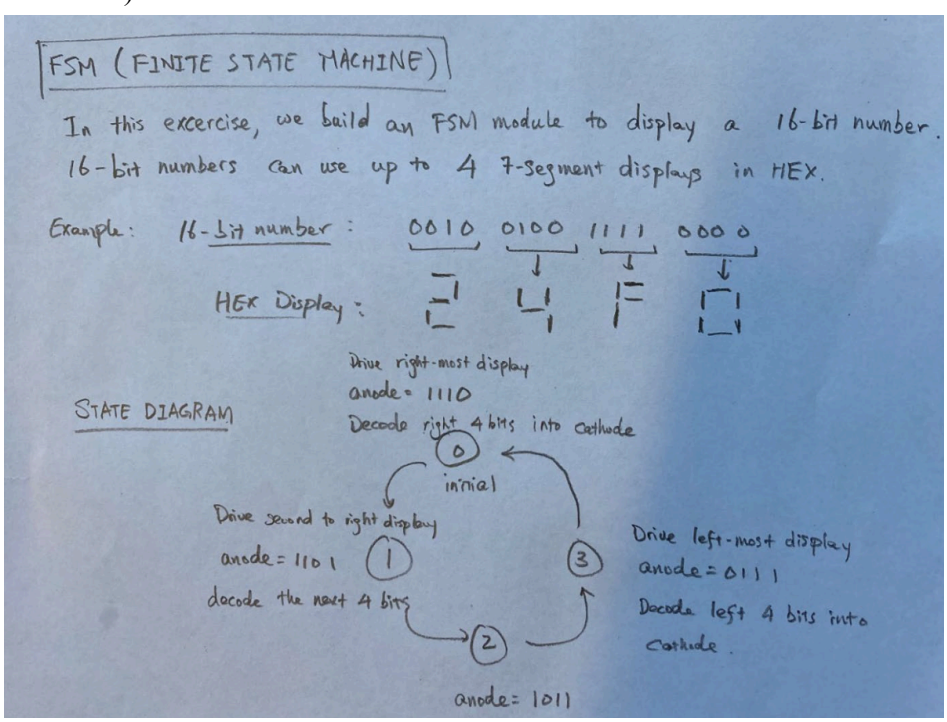
**Please look at Appendix A of the Verilog tutorial to see how 7-segment displays work.** There are 7 cathode wires connected to the 7 segments, and 8 anode wires indicating which of the 8 displays to use. **Both anodes and cathodes are active low!**

All 7-segment displays share the same cathode wires, so how do we drive multiple 7-segment displays simultaneously? The solution is to design an FSM to drive each display in a round-robin fashion. The FSM must be driven by a clock signal.

(a) Fill in the code for *fsm.v* according to the inline comments, reference the figure here. We have three states in the FSM, one for each display. At the positive edge of the input clock, change the state according to the diagram. In each state, set *anode* and *four_bit_number*.

(i) **WARNING**: The state machine (FSM) in the always block should only worry about setting *four_bit_number*, which is the 4-bit number we want to display. Do not set the cathodes directly in the always block. The decoder you instantiated at the beginning of the *fsm* module sets the cathodes.

(ii) Test your FSM code by running *fsm_test.v*. Make sure your anodes and cathodes are correct before proceeding (at least make sure all signals are green, i.e. being driven).




**Step 3**: Putting it together to build a counter.

Now we need to put everything together. Here's a problem: the 100 MHz clock is too fast for the 7-segment display FSM. The latency inside the display is high. We need another clock signal that's fast enough so that we can't visibly see the displays being individually updated, but slow enough so that the display hardware can catch up. 1 kHz is good for this.

(a) Fill in the code of *faster_clock_divider.v*. The implementation is identical to clock_divider.v, except now we're generating a 1kHz signal instead of a 1 Hz signal. Note that you only need to change one number.
(b) Fill in the code of *counter.v* according to the inline comments. Remember that the 1 Hz signal is the output of the *clock_divider*, while the 1 kHz signal is the output of the *faster_clock_divider*.
(c) Use the provided *counter_test.v* to check your code.
(d) Use the provided *constraints.xdc* file to synthesize and implement the design. Remember to set *counter.v* as the top-level module!
(e) Program the board and watch it work.

**Common mistakes**

(a) Remember that you are writing hardware description code, not python …
    (i) This won't work: *output = my_module_name(input)*
    (ii) This will work: my_module_name *my_instance_name(input, output)*
(b) Remember to wrap blocks of code with begin ... end